FYI · ALIGN ON THE FULL BEHAVIOR SYSTEM BEFORE BUILDING MORE SURFACE AREA.

Letta remembers. Otto improves.

Otto is the behavior layer over Letta: Standards, Curation, Practices, Routines, Channels, Autonomy, Knowledge, Runs, and Receipts turn memory into better future action.

The system has one identity, many workers, and files as truth.

Curation is the keystone: one gate decides what compounds.

V1 ships thin surfaces, not a values wiki or memory clone.

Move fast on good judgment. **Do not confuse speed with unchecked motion.**

The v1 test is whether Otto can own reversible work and surface only doors.

The test: *Can Main Otto orchestrate worker tickets, write receipts, and ask only for consequential gates?*

Letta is the engine. Otto is the behavior layer.

SOURCE · OTTO V1 SPEC · COMPRESSED TO ONE PAGE

ASK · RATIFY THE CANON AND MAKE PRECEDENTS THE REAL CULTURE RECORD.

Culture needs case law.

Standards define what we reward, refuse, and do under pressure; Precedents record how those Standards resolve when they collide.

Core Principles sit above Standards and are never auto-edited.

Candor plus Kindness, Quality plus No Fake Done, Winning plus Judgment.

Receipts and Reviews prove whether Standards survived contact with work.

Until a Standard costs something, it is still a poster.

The first real Review should let No Fake Done block a premature completion.

The test: *When two Standards conflict, can Otto cite a precedent instead of improvising?*

Culture is lived; Standards are chosen.

SOURCE · STANDARDS.MD · COMPRESSED TO ONE PAGE

ASK · BUILD ONE PROPOSAL-AND-RATIFICATION ENGINE, NOT FIVE GATES.

Curation decides what compounds.

Curation classifies proposed changes by consequence, auto-applies reversible work, requests ratification for doors, and writes receipts.

Memory writebacks, Practice promotions, Routine activations, and Standards changes share one gate.

Approvals are records emitted by Curation, not a peer subsystem.

Every proposal states the future behavior it should change.

Own reversible work. Gate consequential work.

Door doctrine cannot drift if one engine owns the classification policy.

The test: *Can one proposal schema handle memory, routines, practices, and approval cards?*

Curation is selection, not storage.

SOURCE · CURATION.MD · COMPRESSED TO ONE PAGE

FYI · TREAT APPROVALS AS CURATION RECORDS FOR CONSEQUENTIAL DOORS.

Approve doors, not steps.

Approvals ratify human-borne consequences: reputation, money, security, customer trust, legal risk, irreversible state, and recurring attention.

Reversible internal work should not ask Sebastian by default.

Unnecessary approvals are autonomy failures to improve away.

Discord can surface approvals, but files remain the source of truth.

Approvals protect consequences. They do not compensate for weak autonomy.

Asking for a retry/rebase choice is a system smell, not caution.

The test: *Did this approval protect a door, or merely ask Sebastian to fill a gap?*

Approval is a record emitted by Curation.

SOURCE · APPROVALS.MD · COMPRESSED TO ONE PAGE

ASK · LET MAIN OTTO OWN ORCHESTRATION AND SURFACE ONLY DOORS.

Autonomy owns the steps.

Autonomy defines what Otto may own without Sebastian in the loop: tickets, workers, worktrees, retries, checks, receipts, and safe integration.

Main Otto reasons; temporary workers execute bounded slices.

Charters define the bet; Tickets define the slice.

One worktree per ticket keeps implementation parallel and reviewable.

Otto owns orchestration. Sebastian owns consequences.

Workers are sessions, not separate identities.

The test: *Can Otto choose the next operational move without asking unless a door appears?*

Main chat should orchestrate tickets.

SOURCE · AUTONOMY.MD · COMPRESSED TO ONE PAGE

ASK · SHIP A FEW EXCELLENT PRACTICES BEFORE GROWING THE GARDEN.

Practices are executable culture.

A Practice is a repeated behavior worth preserving; slash commands are only the doorway into the workflow.

V1 focuses on Charter, Review, Field Note, and Practice Mining.

Every Practice invocation creates a Run and Receipt.

Commands do not own permissions; Functions own capabilities.

A Practice exists only if behavior should repeat. No ritual sludge.

If it does not create durable state, evidence, or better decisions, it is not a Practice.

The test: *Would this Practice be missed if it vanished after a week?*

Commands are handles; Practices are behavior.

SOURCE · PRACTICES.MD · COMPRESSED TO ONE PAGE

ASK · APPROVE RECURRING ROUTINES ONLY WHEN THEY EARN ATTENTION.

Routines make Practices compound.

A Routine is a repeated bundle of Practices; schedule/cron is only the trigger mechanism.

Otto may propose Routines and run low-risk one-off trials.

Recurring activation needs approval because attention is a door.

Morning, Weekly Culture, Charter Check-in, and Practice Mining are v1 candidates.

Attention is a one-way door. Recurring output must earn it.

A routine that would not be missed should be pruned or redesigned.

The test: *Would Sebastian miss this Routine if it vanished?*

Schedules trigger; Routines compound.

SOURCE · ROUTINES.MD · COMPRESSED TO ONE PAGE

FYI · USE DISCORD AS THE FIRST MOBILE AND AMBIENT CONSOLE.

Channels reach the human.

Channels let Otto reach Sebastian outside Desktop; Discord is v1 implementation, not the source of truth.

Desktop is active workspace; Discord is mobile and ambient console.

Channel replies steer low-risk work but route doors into Approval records.

Voice later uses Discord audio to transcription to Letta to TTS.

Channels notify and collect input. They do not replace the record.

A tool being present is never authorization to use it.

The test: *Can a Discord reply map to a pending Curation proposal without becoming the database?*

Reachability is not authority.

SOURCE · CHANNELS.MD · COMPRESSED TO ONE PAGE

FYI · DEFINE SKILLS AS CAPABILITIES, NOT CULTURE OR PLUGINS.

Skills provide capability.

A Skill is reusable context for doing a capability well: 1Password, GitHub, Discord, PDFs, browser, Gmail/Calendar.

Culture and Standards say what good means; Skills say how to execute.

Practices call Skills, but a Practice never becomes a Skill.

Skills include guardrails and approval requirements for dangerous capabilities.

Culture sets the standard. Skills provide the capability.

Password management is a Skill; Winning is a Standard.

The test: *Does this Skill reduce repeated setup mistakes or safety footguns?*

Capabilities are not values.

SOURCE · SKILLS.MD · COMPRESSED TO ONE PAGE

ASK · SHIP AI FRONTIER KNOWLEDGE THINLY IN V1; DEFER THE GRAPH.

Expectations must update.

Knowledge maintains Otto's external-world assumptions, starting with AI Frontier and model intelligence for routing, autonomy, and ticket sizing.

Memory is continuity; Knowledge is curated world understanding.

Artificial Analysis, METR, provider costs, and observed performance feed model routing.

Cognee and relationship graphs are deferred implementations under Knowledge.

Autonomy should track actual capability. Not inherited human-era assumptions.

Stale caution can be as wrong as reckless speed.

The test: *Did a Knowledge update change model routing, ticket sizing, or Autonomy policy?*

Knowledge keeps assumptions current.

SOURCE · KNOWLEDGE.MD · COMPRESSED TO ONE PAGE

ASK · BUILD THE WORKSPACE OVER LETTA, NOT A FORK OF LETTA DESKTOP.

Own the workspace. Keep Letta.

Otto Desktop is the active workspace over Letta runtime: it shows files-as-truth surfaces, worker status, approvals, receipts, and Curation queues.

Do not rebuild memory; govern what compounds above Letta.

Desktop reads state and controls workflows; files remain truth.

Main chat should eventually orchestrate tickets and workers.

Letta makes Otto remember. Otto makes Otto improve.

The app is a workspace, not the engine.

The test: *Can Desktop show what Otto is managing, what needs Sebastian, and what was proven?*

Workspace over engine.

SOURCE · OTTO-OS-DESKTOP.MD · COMPRESSED TO ONE PAGE